

Evidencias EV09 — FastAPI con SQLAlchemy

(persistencia de datos)

Guia: GA1-220501096-01-AA1-EV09 **Aprendiz:** Daniel Roman **Proyecto:** device_systems (v3.0.0) — código en `../device_systems/` **Repositorio:** <https://github.com/daniel-roman345/python-avanzado->

1. Estructura del proyecto

```
Windows PowerShell - estructura del proyecto

PS D:\python avanzado\ev09\device_systems> tree /F

device_systems
├── app
│   ├── database
│   │   ├── __init__.py
│   │   └── connection.py
│   ├── dependencies
│   │   ├── __init__.py
│   │   └── database_dependency.py
│   ├── models
│   │   ├── __init__.py
│   │   └── user_model.py
│   ├── routes
│   │   ├── __init__.py
│   │   └── user_routes.py
│   ├── schemas
│   │   ├── __init__.py
│   │   └── user_schema.py
│   ├── services
│   │   ├── __init__.py
│   │   └── user_service.py
│   ├── __init__.py
│   └── main.py
├── .gitignore
├── README.md
├── requirements.txt
└── seed.py
```

2. Base de datos generada

Tabla `users` creada por SQLAlchemy en `device_systems.db`, con sus constraints (`NOT NULL`, `PRIMARY KEY`), sus índices (`ix_users_email`, `ix_users_id`) y los registros almacenados.

```
Windows PowerShell - base de datos device_systems.db

PS D:\python avanzado\ev09\device_systems> sqlite3 device_systems.db

sqlite> .tables
users

sqlite> .schema users
CREATE TABLE users (
  id INTEGER NOT NULL,
  name VARCHAR(60) NOT NULL,
  email VARCHAR(120) NOT NULL,
  role VARCHAR(20) NOT NULL,
  is_active BOOLEAN NOT NULL,
  created_at DATETIME NOT NULL,
  PRIMARY KEY (id)
);

-- indices
ix_users_email
ix_users_id

sqlite> PRAGMA table_info(users);
cid | name      | type      | notnull | dflt_value | pk
-----+-----+-----+-----+-----+-----
0   | id        | INTEGER   | 1       | None       | 1
1   | name      | VARCHAR(60) | 1       | None       | 0
2   | email     | VARCHAR(120) | 1       | None       | 0
3   | role      | VARCHAR(20) | 1       | None       | 0
4   | is_active | BOOLEAN    | 1       | None       | 0
5   | created_at | DATETIME   | 1       | None       | 0

sqlite> SELECT * FROM users;

id | name      | email      | role   | activo | created_at
---+-----+-----+-----+-----+-----
1  | Ana Perez | ana@sena.edu.co | admin | True   | 2026-09-12 17:28:28.082396
2  | Carlos A Gomez | carlos.g@sena.edu.co | admin | False  | 2026-09-12 17:28:28.082402
3  | Laura Martinez | laura@sena.edu.co | support | False  | 2026-09-12 17:28:28.082404
```

3. Capturas de Swagger UI

device_systems API

3.0.0

OAS 3.1

[/openapi.json](#)

API REST para la gestion de usuarios del sistema **device_systems**.

Novedades de esta version

- Persistencia real con **SQLAlchemy** sobre **SQLite**.
- Modelo **User** con constraints (**unique** , **nullable=False**).
- Sesion de base de datos inyectada con **Depends(get_db)** .
- CRUD completo, filtros y ordenamiento sobre la base de datos.

Contact Daniel Roman - Aprendiz SENA

Users

CRUD completo del recurso **usuarios** persistido en base de datos.

GET

/users

Listar usuarios

POST

/users

Crear un usuario

GET

/users/{user_id}

Consultar usuario por ID

PUT

/users/{user_id}

Actualizar un usuario por completo

PATCH

/users/{user_id}

Actualizar parcialmente un usuario

DELETE

/users/{user_id}

Eliminar un usuario

Root

Informacion general y estado de la API.

GET

/

Informacion de la API

GET

/health

Estado del servicio

Schemas

HTTPValidationError

>

Expand all

object

MessageResponse

>

Expand all

object

OrdenUsuarios

>

Expand all

string

UserCreate

>

Expand all

object

UserListResponse

>

Expand all

object

UserPatch

>

Expand all

object

UserResponse

>

Expand all

object

UserRole

>

Expand all

string

UserUpdate

>

Expand all

object

ValidationError

>

Expand all

object

4. Evidencia de prueba de cada endpoint

4.1 POST /users — crear usuario en base de datos (201)

POST /users Crear un usuario

Registra un usuario en la base de datos validando los datos con Pydantic y comprobando que el correo no exista (constraint unique).

Parameters

No parameters

Cancel

Reset

Request body required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8009/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pedro Ruiz",
    "email": "pedro@sena.edu.co",
    "role": "user",
    "is_active": true
  }'
```

Request URL

http://127.0.0.1:8009/users

Server response

Code

Details

201

Response body

```
{
  "id": 4,
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true,
  "created_at": "2026-09-12T17:49:30.567723"
}
```

Download

Response headers

```
content-length: 129
content-type: application/json
date: Sat, 12 Sep 2026 17:49:30 GMT
location: /users/4
server: uvicorn
x-api-version: 3.0
x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:30.7792" }</pre>	No links
400	Solicitud invalida	No links
422	Datos invalidos	No links

Pagina 4 de 16

4.2 GET /users — listar usuarios

GET /users Listar usuarios

Consulta los usuarios almacenados en la base de datos. Permite filtrar por **role** e **is_active** y ordenar por **name** o **created_at**.

Parameters

Name

Description

role

Filtra por rol

string | (string | null) (query)

--

is_active

Filtra por estado

boolean | (boolean | null) (query)

--

order_by

Campo de ordenamiento

string (query)

name

desc

Orden descendente

boolean (query)

false

Execute

Clear

Responses

Curl

curl -X 'GET' \ 'http://127.0.0.1:8009/users?order_by=name&desc=false' \ -H 'accept: application/json'

Request URL

http://127.0.0.1:8009/users?order_by=name&desc=false

Server response

Code

Details

200

Response body

{ "total": 3, "data": [{ "id": 1, "name": "Ana Perez", "email": "ana@sona.edu.co", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:28:08Z396" }, { "id": 2, "name": "Carlos Gomez", "email": "carlog@sona.edu.co", "role": "support", "is_active": true, "created_at": "2026-09-12T17:28:28Z482" }, { "id": 3, "name": "Laura Martinez", "email": "lauram@sona.edu.co", "role": "user", "is_active": false, "created_at": "2026-09-12T17:28:28Z484" }] }

Response headers

content-length: 413 content-type: application/json date: Sat, 12 Sep 2026 17:49:26 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems x-total-users: 3

Responses

Code

Description

Links

200

Listado de usuarios

No links

Media type

application/json

Content Accept header

Example Value | Schema

{ "total": 3, "data": [{ "id": 1, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:27Z272" }] }

422

Validation Error

No links

Media type

application/json

Example Value | Schema

{ "detail": [{ "loc": ["string",], "msg": "string", "type": "string", "ctx": { "type": "string", "ctx": {} } }] }

Pagina 5 de 16

4.3 GET /users/{user_id} — consultar por ID

GET /users/{user_id} Consultar usuario por ID

Obtiene un usuario de la base de datos por su identificador.

Parameters

Name	Description
user_id * required	Identificador del usuario
integer (path)	1
	minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8009/users/1' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8009/users/1
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Ana Perez", "email": "ana@sena.edu.co", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:28:28.002396" }</pre> Download Response headers <pre>content-length: 127 content-type: application/json date: Sat, 12 Sep 2026 17:49:37 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:38.5752" }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

4.4 GET /users?role=admin — filtrar por rol

GET /users

Listar usuarios

Consulta los usuarios almacenados en la base de datos. Permite filtrar por **role** e **is_active** y ordenar por **name** o **created_at**.

Parameters

Cancel

Name	Description
role string (string null) (query)	Filtra por rol admin
is_active boolean (boolean null) (query)	Filtra por estado --
order_by string (query)	Campo de ordenamiento name
desc boolean (query)	Orden descendente false

ExecuteClear

Responses

Curl

curl -X 'GET' \ 'http://127.0.0.1:8009/users?role=admin&order_by=name&desc=false' \ -H 'accept: application/json'

Request URL

http://127.0.0.1:8009/users?role=admin&order_by=name&desc=false

Server response

Code	Details
200	Response body <pre>{ "total": 1, "data": [{ "id": 1, "name": "Ana Perez", "email": "ana@ana.edu.co", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:28:28.083396" }]}</pre> Download Response headers content-length: 148 content-type: application/json date: Sat, 12 Sep 2026 17:49:51 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems x-total-users: 1

Responses

Code	Description	Links
200	Listado de usuarios Media type application/json Controls Accept header Example Value Schema <pre>{ "total": 0, "data": [{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:53.0362" }]}</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>	No links

4.5 GET /users?is_active=true — filtrar por estado

GET /users Listar usuarios

Consulta los usuarios almacenados en la base de datos. Permite filtrar por **role** e **is_active** y ordenar por **name** o **created_at**.

Parameters

Cancel

Name	Description
role	Filtra por rol
string (string null) (query)	--
is_active	Filtra por estado
boolean (boolean null) (query)	true
order_by	Campo de ordenamiento
string (query)	name
desc	Orden descendente
boolean (query)	false

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8009/users?is_active=true&order_by=name&desc=false' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8009/users?is_active=true&order_by=name&desc=false
```

Server response

CodeDetails

200

Response body

```
{
  "total": 3,
  "data": [
    {
      "id": 1,
      "name": "Ana Perez",
      "email": "ana@sona.edu.co",
      "role": "admin",
      "is_active": true,
      "created_at": "2026-09-12T17:28:28.002396"
    },
    {
      "id": 2,
      "name": "Carlos Gomez",
      "email": "carlos@sona.edu.co",
      "role": "support",
      "is_active": true,
      "created_at": "2026-09-12T17:28:28.002402"
    },
    {
      "id": 4,
      "name": "Pedro Ruiz",
      "email": "pedro@sona.edu.co",
      "role": "user",
      "is_active": true,
      "created_at": "2026-09-12T17:49:30.567223"
    }
  ]
}
```

Response headers

```
content-length: 414
content-type: application/json
date: Sat, 12 Sep 2026 17:49:56 GMT
server: uvicorn
x-api-version: 3.0
x-app-name: device_systems
x-total-users: 3
```

Responses

Code	Description	Links
200	Listado de usuarios	No links
Media type: application/json		
Content Accept header:		
Example Value Schema		
<pre>{ "total": 0, "data": [{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:56.729Z" }] }</pre>		
422	Validation Error	No links
Media type: application/json		
Example Value Schema		
<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "value": "string", "ctx": {} }] }</pre>		

Pagina 8 de 16

4.6 PUT /users/{user_id} — actualizacion completa

PUT

/users/{user_id} Actualizar un usuario por completo

Reemplaza todos los campos del usuario en la base de datos.

Parameters

Cancel

Reset

Name	Description
user_id required	Identificador del usuario
integer (path)	2
	minimum: 1

Request body required

application/json

Edit Value

Schema

```
{  "name": "Carlos A Gomez",  "email": "carlos.g@sena.edu.co",  "role": "admin",  "is_active": false}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  'http://127.0.0.1:8009/users/2' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "name": "Carlos A Gomez",  "email": "carlos.g@sena.edu.co",  "role": "admin",  "is_active": false}'
```

Request URL

```
http://127.0.0.1:8009/users/2
```

Server response

Code	Details
200	Response body <pre>{ "id": 2, "name": "Carlos A Gomez", "email": "carlos.g@sena.edu.co", "role": "admin", "is_active": false, "created_at": "2026-09-12T17:28:28.082402Z"}</pre> Download Response headers <pre>content-length: 138 content-type: application/json date: Sat, 12 Sep 2026 17:58:06 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario actualizado	No links
	Media type application/json Controls Accept header: Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:58:00.798Z"}</pre>	
400	Solicitud invalida	No links
404	Usuario no encontrado	No links
422	Validation Error	No links
	Media type application/json Controls Accept header: Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>	

4.7 PATCH /users/{user_id} — actualizacion parcial

PATCH /users/{user_id} Actualizar parcialmente un usuario

Modifica solo los campos enviados. Si el cuerpo llega vacío responde 400 Bad Request.

Parameters

Name	Description
user_id required	Identificador del usuario
integer (path)	
	minimum: 1

Cancel

Request body required

application/json

Edit Value Schema

```
{  "role": "support"}
```

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \
  http://127.0.0.1:8089/users/3 \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "role": "support"
  }'
```

Request URL

http://127.0.0.1:8089/users/3

Server response

Code	Details
200	Response body <pre>{ "id": 3, "name": "Laura Martinez", "email": "lauramartinez@edu.co", "role": "support", "is_active": false, "created_at": "2026-09-12T17:28:28.082404"}</pre> Download Response headers <pre>content-length: 127 content-type: application/json date: Sat, 12 Sep 2026 17:50:03 GMT server: unicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario actualizado parcialmente	No links
Media type application/json Controls Accept header Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:50:04.9202"}</pre>		
400	Solicitud invalida	No links
404	Usuario no encontrado	No links
422	Validation Error	No links
Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }]}</pre>		

Pagina 10 de 16

4.8 DELETE /users/{user_id} — eliminar

DELETE

/users/{user_id}

Eliminar un usuario

⌵

Elimina el usuario de la base de datos.

Parameters

Cancel

Name	Description
user_id * required	Identificador del usuario
integer (path)	4 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8009/users/4' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8009/users/4
```

Server response

Code	Details
200	Response body <pre>{ "message": "Usuario 4 eliminado correctamente" }</pre> Download Response headers <pre>content-length: 47 content-type: application/json date: Sat, 12 Sep 2026 17:50:08 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario eliminado correctamente Media type application/json Controls Accept header. Example Value Schema <pre>{ "message": "Usuario eliminado correctamente" }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

4.9 Verificacion: el usuario eliminado ya no existe

GET

/users/{user_id}

Consultar usuario por ID

Obtiene un usuario de la base de datos por su identificador.

Parameters

Cancel

Name	Description
user_id * required	Identificador del usuario
integer (path)	4 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8009/users/4' \
-H 'accept: application/json'
```

Request URL

http://127.0.0.1:8009/users/4

Server response

Code	Details
404	Error: Not Found Response body <pre>{ "error": true, "message": "Usuario no encontrado", "status_code": 404, "path": "/users/4" }</pre> Response headers <pre>content-length: 84 content-type: application/json date: Sat, 12 Sep 2026 17:50:12 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:50:13.021Z" }</pre>	

 No links || 404 | Usuario no encontrado | No links |
| 422 | Validation Error Media type application/json Example Value | Schema ``` { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] } ``` | No links |

5. Evidencia de errores controlados

5.1 Email duplicado — 400 Bad Request

POST

/users

Crear un usuario

Registra un usuario en la base de datos validando los datos con Pydantic y comprobando que el correo no exista (constraint unique).

Parameters

No parameters

Cancel

Reset

Request body

required

application/json

Edit Value

Schema

```
{
  "name": "Pedro Ruiz",
  "email": "pedro@sena.edu.co",
  "role": "user",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8009/users' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pedro Ruiz",
    "email": "pedro@sena.edu.co",
    "role": "user",
    "is_active": true
  }'
```

Request URL

http://127.0.0.1:8009/users

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "error": true,
  "message": "El correo ya esta registrado",
  "status_code": 400,
  "path": "/users"
}
```

Download

Response headers

```
content-length: 89
content-type: application/json
date: Sat, 12 Sep 2026 17:49:34 GMT
server: uvicorn
x-api-version: 3.0
x-app-name: device_systems
```

Responses

Code	Description	Links
201	Usuario creado	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:34.905Z" }</pre>		

| 400 | Solicitud invalida | No links |
| 422 | Datos invalidos | No links |

5.2 Usuario no encontrado — 404 Not Found

GET

/users/{user_id}

Consultar usuario por ID

Obtiene un usuario de la base de datos por su identificador.

Parameters

Cancel

Name	Description
user_id required	Identificador del usuario
integer (path)	99 minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8009/users/99' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8009/users/99

Server response

Code	Details
404	Error: Not Found Response body <pre>{ "error": true, "message": "Usuario no encontrado", "status_code": 404, "path": "/users/99" }</pre> Response headers <pre>content-length: 85 content-type: application/json date: Sat, 12 Sep 2026 17:49:42 GMT server: uvicorn x-api-version: 3.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Usuario encontrado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:49:42.741Z" }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

6. Diferencia entre modelo SQLAlchemy y schema Pydantic

	Modelo SQLAlchemy (User)	Schema Pydantic (UserCreate , UserResponse ...)
Para que sirve	Representa la tabla de la base de datos	Representa el contrato HTTP de entrada/salida
Donde vive	app/models/user_model.py	app/schemas/user_schema.py
Que declara	Columnas, tipos SQL, constraints, indices	Campos, tipos Python, validaciones y ejemplos
Quien valida	El motor de base de datos (unique , nullable)	Pydantic, antes de llegar a la base de datos
Ejemplo de regla	email con unique=True	email: EmailStr con formato valido
Si falla	IntegrityError → traducido a 400	422 Unprocessable Entity

El puente entre ambos es `model_config = ConfigDict(from_attributes=True)`, que permite construir el schema directamente desde el objeto de la base de datos:

```
return UserResponse.model_validate(usuario)
```

7. Resumen de pruebas

#	Prueba	Esperado	Resultado
1	Crear usuario valido	201	OK
2	Crear usuario con email repetido	400	OK
3	Listar usuarios	200	OK
4	Consultar usuario por ID	200	OK
5	Consultar usuario inexistente	404	OK
6	Filtrar por rol	200	OK
7	Filtrar activos	200	OK
8	Actualizar con PUT	200	OK
9	Actualizar con PATCH	200	OK
10	Eliminar con DELETE	200	OK
11	Validar que el eliminado ya no exista	404	OK

8. Reflexion final sobre la importancia de la persistencia

Trabajar con datos en memoria servia para entender los verbos HTTP, pero cualquier reinicio borraba todo: la API no era utilizable en un escenario real. Al conectar SQLAlchemy, `device_systems` paso a tener un estado que sobrevive a la aplicacion.

Lo que mas me quedo claro es la separacion entre modelo y schema: el modelo describe como se guarda la informacion y el schema como se comunica la API con el cliente. Gracias a `from_attributes=True` convertir uno en otro es directo, y puedo exponer `created_at` sin que el cliente lo envíe nunca.

Tambien entendi el valor de tener dos capas de validacion: Pydantic rechaza los datos mal formados antes de tocar la base de datos, y los constraints protegen la integridad aunque el error venga de otro lado; por eso agregue un manejador de `IntegrityError` que traduce esa violacion a un `400` claro en vez de un error 500.

Finalmente, la dependencia `get_db()` con `yield` me mostro como FastAPI administra recursos: abre la sesion, la entrega al endpoint y la cierra siempre, incluso si ocurre una excepcion.

Documento de evidencias generado desde el repositorio python-avanzado- · device_systems